

Innovation en optimisation de particules

Groupe 07

Doryan JOVANOVIĆ

Alice MERAUD

Anitta KANGPAPHANH

1 Description du problème et de l'innovation

1.1 Le problème

Nous étudions un environnement contenant n particules dans une zone de taille $w \times h$, avec un pas de temps dt et un rayon de voisinage r . Les particules se déplacent, rebondissent sur les bords et se repoussent localement. Notre objectif est de choisir leurs positions et vitesses initiales pour minimiser la distance maximale entre deux particules à la première itération qui dépasse T .

1.2 L'innovation

Notre méthode consiste à tester plusieurs configurations initiales, puis à garder la meilleure. Une configuration en grille et plusieurs configurations aléatoires sont générées. Dans tous les cas, les vitesses sont dirigées vers le centre de l'environnement pour favoriser le regroupement. Après simulation jusqu'au temps T , on conserve la configuration qui minimise cette valeur.

2 Protocole expérimental

2.1 Mesures effectuées

Les mesures sont obtenues directement via les fonctions du module `evaluation.c` :

- **Temps d'exécution** : `measureExecutionTime(env, T, &temps)`
- **Distance maximale** : `findMaxDist(env, &distmax)`
- **Distance moyenne** : `averageDistance(env, &distAverage)`

2.2 Paramètres de simulation

Chaque environnement est créé par : `create_environnement(n, w, h, R, dt)`

avec les valeurs par défaut : $n = 20$, $w = h = 50$, $R = 1.0$, $dt = 0.2$, $T = 2.0$

2.3 Procédure expérimentale

1. Création de l'environnement : `env e = create_environnement(...)`
2. Exécution de l'algorithme et mesure du temps : `measureExecutionTime(e, T, &temps)`
3. Calcul des distances : `averageDistance(e, &distAverage)` et `findMaxDist(e, &distmax)`
4. Libération mémoire : `free_environnement(e)`

2.4 Cas testés

Cas limites

- $n = 0$: `casLimiteParticules1(&temps, &distAverage, &distmax)`
- $n = 1$: `casLimiteParticules2(&temps, &distAverage, &distmax)`
- $R \geq \max(w, h)$: `casLimiteRayon(&temps, &distAverage, &distmax)`
- $T = 0$: `casLimitenbIteration(&temps, &distAverage, &distmax)`

Cas généraux

Chaque test fait varier un paramètre unique :

- Variation du nombre de particules (n) : `casParticule(particleCount, executionTime, averageDist, maxDistance)`
- Comparaison carré / rectangle de même aire ($cote$) : Compare un carré et un rectangle de même aire. Choisit un w aléatoire, calcule $h = \frac{cote^2}{w}$ pour garder $w \times h = cote^2$.
`casenvironnementdiffCarrerectangle(sideLength, squareExecutionTime, squareAverageDist, squareMaxDistance, rectExecutionTime, rectAverageDist, rectMaxDistance)`

- Variation du nombre d'itérations (T) : `casnbIteration(T, &temps, &distAverage, &distmax)`
- Variation du rayon d'interaction (R) : `casRayon(R, &temps, &distAverage, &distmax)`
- Variation du pas de temps (dt) : `casDureIteration(dt, &temps, &distAverage, &distmax)`

3 Résultats de l'évaluation

3.1 Complexité et temps de calcul

Nos tests confirment que l'algorithme a une complexité quadratique par rapport au nombre de particules. Sur la [figure 1](#), on voit bien que le temps d'exécution montre une croissance quadratique. C'est un résultat logique puisqu'on compare chaque particule avec toutes les autres, ce qui donne du $O(n^2)$.

Du côté du temps de simulation (T), l'évolution est parfaitement linéaire ($O(T)$), ce qui montre que le programme reste stable au fil des itérations. On a aussi remarqué en comparant un environnement carré et un environnement rectangulaire que la forme de l'espace ne change rien à la vitesse d'exécution.

3.2 Évaluation du critère (Distance maximale)

Le but de notre lot était de minimiser la distance maximale entre deux particules. En regardant la [figure 2](#), on constate que l'algorithme remplit bien son rôle : en laissant tourner la simulation plus longtemps (quand T augmente), la distance max baisse bien, passant de 42.5m au début à environ 23m.

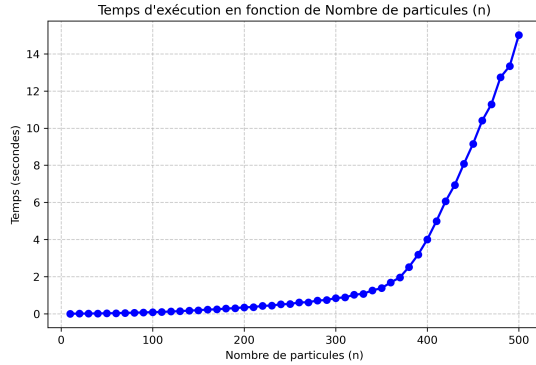


FIGURE 1 – Temps d'exécution selon n .

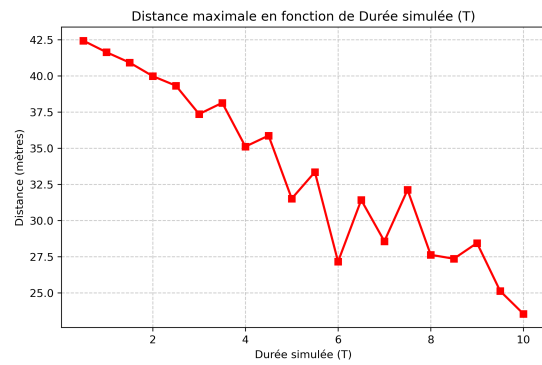


FIGURE 2 – Baisse de la distance max selon T .

On a aussi noté quelques autres comportements intéressants pendant les tests :

- **Saturation de l'espace** : En ajoutant beaucoup de particules, la distance maximale finit par bloquer à une certaine limite (la diagonale de l'environnement) car la boîte devient juste trop remplie.
- **Impact du rayon (R)** : Si on augmente le rayon, les particules se repoussent de plus loin. À partir d'un rayon de 8m, presque toutes les particules interagissent ensemble tout le temps, ce qui fait saturer le temps de calcul à son maximum.
- **Impact des proportions** : Pour les rectangles générés avec une largeur très fine, la distance max peut monter jusqu'à 900m comparé à un carré de même surface. C'est normal vu la forme en étroite de l'environnement.

4 Conclusion

Pour conclure, l'innovation qu'on a mise en place fonctionne : la distance maximale entre les particules diminue bien au fil du temps, ce qui répond directement à l'objectif du lot. Par contre, nos évaluations montrent bien la limite du $O(n^2)$ pour la complexité. L'algorithme tourne très vite pour un petit nombre de particules, mais il aura beaucoup de mal à passer à l'échelle si on veut simuler de très grands environnements en temps réel.